

Guidelines for Submission to MICRO 2026

MICRO 2026 Submission #NaN – Confidential Draft – Do NOT Distribute!!

MICRO 2026 Submission #NaN – Confidential Draft – Do NOT Distribute!!

Abstract

Keywords

Do, Not, Use, This, Code, Put, the, Correct, Terms, for, Your, Paper

1 Introduction

Memory-level parallelism (MLP) is important for efficient out-of-order (OoO) execution and overcoming the memory wall in processor performance. Memory dependence prediction is a speculative technique used in OoO processors that aims to enhance total MLP by predicting the stores on which a given load instruction will depend, as well as identifying memory-independent loads. Smaller cores utilising smaller predictors can often struggle with a large number of false dependencies, which unnecessarily stalls loads on non-dependent stores. This is due to hash collisions in small predictor tables, or simple predictor algorithms that cannot capture varying dependency behaviour. The Store Sets predictor [?] is exhibits both of these issues, and variations of the algorithm are found in real processors today [?]. Whereas modern predictors like PHAST [?] have pushed the SotA to address these issues via techniques like associative tagging and context based predictions, they place higher area, energy and complexity demands on core design, which is often not viable in smaller cores. At the same time, while simply growing a Store Sets-like predictor would offset the hash collisions, this benefit struggles to outperform an iso-area comparison to scaling other components. As such, memory dependence predictors in small cores typically remain small and inefficient.

At the same time, the importance of such area-constrained cores has grown over time. Modern systems increasingly deploy heterogeneous processors containing both performance and efficiency-oriented core designs, combining larger high-performance cores with small energy-efficient (but still out-of-order) cores, as seen in ARM big.LITTLE systems and both Apple's & Intel's performance and efficiency cores. Efficiency-focused cores are, of course, also found in power-constrained systems such as smartphones, tablets, smartwatches and other edge systems. These cores are more and more performing demanding general purpose computation while striving to consume as little energy as possible. As they grow more performance-sensitive, finding optimisations which don't also incur additional area, power or complexity costs becomes more pressing.

This paper builds on an observation demonstrated by various prior works that a considerable portion of load instructions in general-purpose workloads rarely or never exhibit in-flight memory dependencies [???]. This is exploited to reduce false dependencies in memory dependence prediction via a profile-guided software co-design at **zero hardware cost**. Our technique labels frequently memory-independent loads by their opcode, causing them to bypass MDP queries and always schedule as early as possible. We show that the vast majority of dynamic loads can avoid querying the

MDP while maintaining or improving IPC, as well as provide a potential power saving.

1.1 Contributions

This paper makes the following contributions:

- Implements a PGO technique to label memory independent loads to the OoO core at no-cost communication or hardware overhead.
- Implements a modified Gem5 to simulate labelled loads against different MDP algorithms and core size configurations, as well as extends McPat to provide power usage estimations of the MDP unit.
- Evaluates Spec2017 IntSpeed and demonstrates on average: 71% of MDP load queries are redundant, IPC improves by 1.47% & MDP power reduces by 12% on the smallest core, whereas the largest core maintains IPC and reduces MDP power by 42%.
 - Additionally demonstrates a 3.2% IPC gain on highly constrained cores that lack any memory dependence prediction.
 - Additionally demonstrates a x% IPC gain when modelling a limited number of MDP read ports.
- (currently only includes 2 benchmarks) Evaluates Spec2017 FloatSpeed and demonstrates on average: 87% of MDP load queries are redundant, IPC improves by 0.1% & MDP power reduces by 6.5% on the smallest core, whereas the largest core maintains IPC and reduces MDP power by x%.
 - Additionally demonstrates a 16.5% IPC gain on highly constrained cores that lack any memory dependence prediction.
 - Additionally demonstrates a x% IPC gain when modelling a limited number of MDP read ports.

2 Background

This section outlines the fundamental concepts in memory-level parallelism for out-of-order execution. It then introduces Store Sets as a foundational memory dependence predictor algorithm, a more modern variation found in the XiangShan processor, and lastly the current state of the art in the PHAST predictor.

2.1 Loads in Out-of-Order Execution

The key structure in memory-level parallelism is the load-store queue (LSQ), used to hold in-flight memory operations and perform memory disambiguation. When load instructions dispatch, they are inserted into the load queue (LQ), and query the memory dependence predictor (MDP) for a predicted store(s) to wait on before execution. Once their source operands are ready and all predicted dependent stores have executed, loads search backwards through the store queue (SQ) for store-forwarding opportunities. If no matching addresses are found, the loads access cache to get their data.

MICRO 2026, Athens, Greece

2026. ACM ISBN 978-X-XXXX-XXXX-X/XX/XX

<https://doi.org/XXXXXXXX.XXXXXXX>

When a load searches the SQ, not all earlier stores may have resolved their addresses. As the majority of the time the address will not match with the load, it is often profitable to speculatively ignore these stores and issue the load as soon as possible. When the unresolved store eventually receives its address, it searches forward through the LQ to find loads with matching addresses which have already executed. If a match is found, the load has read stale data and a memory order violation has occurred. The processor must flush all in-flight instructions from the load onwards and re-fetch.

The goal of the MDP is to minimise violation events while maximising the available MLP, allowing memory independent loads to issue as soon as possible and dependent loads to wait only for the necessary stores. When a load is incorrectly stalled on a store that does not resolve to the same address this is called a false dependency. Simple MDP algorithms typically prioritise reducing the number of violations over the number of false dependencies.

2.2 Store Sets

A seminal paper in memory dependence prediction is ‘Memory Dependence Prediction using Store Sets’ [?]. This is the MDP algorithm implemented in the open source Gem5 simulator [?]. Store Sets works by using Store Set IDs (SSID) to track dependent load-store pairs, assigning each instruction the same ID value in the PC-indexed Store Set ID Table (SSIT). The SSIDs are then used to index the Last-Fetched Store Table (LFST), which holds the sequence number of the most recently issued store with an SSID mapping to that LFST entry. Newly issued loads then access the LFST via their SSID to find the store they’re predicted to be dependent on. If a load PC does not map to a valid SSID entry, it is predicted to be memory independent. To forget old dependencies and reduce table pressure, the SSIT and LFST are cyclically cleared after a certain number of issued memory operations.

Store Sets is extremely area efficient, delivering a large portion of available performance with a very small hardware budget. Its small size and simplicity make it well suited to area-constrained processors. However, especially at smaller sizes, Store Sets struggles with false dependencies due hash collisions. When a memory independent load hashes to an existing SSID entry, it is incorrectly made dependent on the corresponding store for that entry. The clear period can be made shorter to offset this, but this comes with the trade-off of a larger number of memory order violations as the predictor must re-learn true dependencies more often. Furthermore, many loads exhibit non-consistent memory dependencies. A load in a loop may be dependent on a store every even iteration, but not on odd iterations. Store Sets has no way to disambiguate these cases, and conservatively predicts the load as dependent on every iteration.

2.3 XiangShan Store Sets

XiangShan is an open-source high-performance RISC-V processor RTL implementation [?]. XiangShan represents the cutting edge in open-source superscalar processor design, and comes with a Gem5 fork modified to closer model the RTL implementation [?]. This Gem5 fork implements a variation of the Store Sets predictor that

offers a closer representation to Store Sets implementations found in real processors. From here on this is referred to as XS Store Sets.

The main optimisation over original Store Sets is using multiple ‘slots’ for each LFST entry, thereby recording multiple store sequence numbers at once. This allows a load to track multiple dependencies with only one SSID. This is useful in situations where a load is dependent on different stores in different program contexts, or may have partial address overlap with multiple stores. XS Store Sets also clears its tables according to the number of elapsed cycles rather than observed memory operations. There are further optimisations on allocation policy which we omit here.

2.4 PHAST

PHAST is a newer predictor which achieves much greater accuracy than Store Sets and other previously proposed MDP algorithms [?]. PHAST uses a TAGE-like structure to record a geometric series of branch history lengths between dependent load-store pairs. On dispatch, load instructions search all history length tables in parallel and selects the dependencies on the longest path (or predicts no dependency if no tables hit). This scheme addresses the previous problems in Store Sets by using multi-way tagged associative entries to reduce hash collisions, and predicting dependencies based on branch context to capture more elaborate dependency patterns. At a storage budget of 14.5KB, PHAST is shown to achieve within 1.5% accuracy of an ideal predictor.

Although PHAST delivers much greater accuracy, it also comes with greater hardware overhead. Though PHAST greatly outperforms Store Sets for iso-area comparisons at larger budgets, it struggles with the smaller budgets most efficiency-focused cores allow for memory dependence prediction. Furthermore, PHAST incurs pipeline complexity by requiring global branch history be made available in the load-store unit, has greater prediction latency, and consumes more power for the same area. This makes PHAST better suited only for larger performance-focused cores that would benefit most from the greater MLP.

3 Related Work

- my original ARCS paper
- store distances paper & comparison to us
- constable
- LLVM store queue search skip paper
- LSQ scalability labelling paper
- software-hardware cooperative memory disambiguation (much more overhead for much less gain, doesn’t address multi-cores, only evaluates floats)

4 Method

- justifying PGO: how PGO is commonly used and static analysis (even SVF) doesn’t work
- where we win (read only/likely stable, hash collisions, resolved stores, occasional dependence)

5 Evaluation

- experimental design
- cpu size selection reasoning
- IPC

- False deps
- Lookups
- Energy
- Violations
- Read ports - PGO & threshold distance scaling
- Prior store distance work comparison
- Constable coverage comparison

This section presents the experimental results using our PGO technique. It highlights the extent of redundant memory dependence predictor queries found in general-purpose workloads and their impact on IPC and power usage. Our results demonstrate that a large portion of predictor activity can be avoided, leading to improvements in performance, power usage, and even that PND load labels can serve as a partial replacement for the memory dependence predictor unit, all with zero hardware overhead. Furthermore, we examine the generalisability of our generated store distance profiles to unseen inputs and provide a comparison to similar prior work [?].

5.1 Experimental Design

We use Gem5 to simulate varying core size configurations to measure impact as components scale, going from a very small core (ROB=64) to a large core (ROB=512). Each core either utilises the XS Store Sets or PHAST predictor, depending on which provides better performance for a realistic area budget at that size. We find that XS Store Sets performs optimally in smaller area allocations, whereas PHAST is more accurate when allowed larger areas. Additionally, we model the v. small configuration to not use a memory dependence predictor at all, and instead conservatively stall loads on all unresolved stores. This shows the impact on highly constrained cores that cannot spare the overhead for memory dependence prediction whatsoever.

The full configuration for each model is detailed in Table 1. All models except the v. small core use a 64KB TAGE-SC-L for branch prediction and a DCPT prefetcher. To better model the constrained size of the v. small core, it employs a tournament branch predictor with 2k local and 8k global entries and a stride prefetcher. We evaluate the Spec2017 Intspeed and Floatspeed benchmarks using up to 10 simpoints, with an interval of 100M instructions and a warm-up of 10M. Gem5 is modified to bypass MDP lookups for labelled loads and to avoid creating new entries in the case of a memory order violation. Violating labelled loads still roll back as normal and do not alter program semantics. The Gem5 results are subsequently processed by an extended McPat, which models XS Store Sets or PHAST as applicable (or upstream McPat for the v. small core).

5.2 Performance

Figure 1 shows the percent changes in IPC across Spec2017 intspeed for each core configuration. The gmean IPC gain ranges from 1.4%, 0.7% and 0% as core size grows. This demonstrates our techniques ability to deliver zero-cost performance gains on area-constrained cores while still maintaining performance on larger cores with more sophisticated MDPs. Examining the gains in the best case (small core size), the IPC gains have an uneven spread across benchmarks, with some seeing large benefits and others next to none. This loosely

Table 1: Parameters of processor components for each simulated model

		V. Small	Small	Med.	Large
Instruction Window	ROB:	64	128	256	512
	IQ:	52	77	154	308
	LQ:	21	38	77	154
	SQ:	12	22	54	108
Pipeline Width	Fetch/Commit:	3	4	6	8
	Issue:	6	8	8	12
L1 Cache	L1D (KB):	16	32	64	64
	L1I (KB):	16	32	32	64
	MSHRs:	4	8	16	32
L2 Cache	L2 (MB):	1	2	4	8
	MSHRs:	8	16	32	64
XS Store Sets	SSIT:	N/A	64	128	N/A
	LFST:		32	32	
	Slots:		2	4	
	Clear:		125k	125k	
PHAST	Rows:	N/A	N/A	N/A	32
	Assoc:				2
	Tag Bits:				8
	Counter				2
	Bits:				2
MDP Read Ports:		N/A	1	2	4

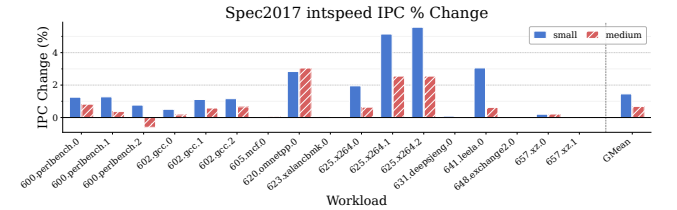


Figure 1: IPC % changes for each CPU model on each intspeed benchmark.

correlates to the magnitude of false dependencies before and after seen in Figure 2. The four largest gainers - 625.x264_s inputs 1 and 2, 641.leela_s and 620.omnetpp_s - all exhibit relatively higher false dependencies per kilo-instruction in the base case, and see this cut down significantly by over 80% when PND labels are used. As the core size increases to the medium configuration, most of these gains are diminished as the larger Store Set tables reduce the base number of hash collisions. A sensitivity analysis on the size of Store Sets on the small configuration results is shown in Section 5.6.

620.omnetpp_s exhibits more false dependencies per kilo-instruction in the medium core despite the larger Store Set size. This is likely because while it doesn't issue the most loads overall, it does have the highest number of dependent loads. Whereas the average across intspeed benchmarks is approximately 2 million dependent loads,

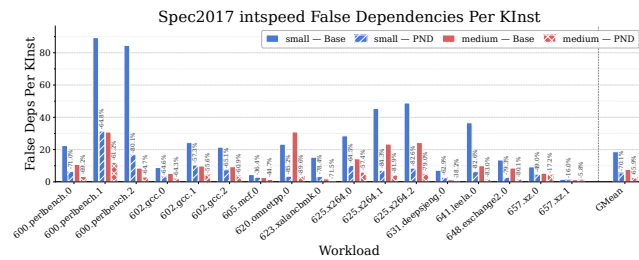


Figure 2: Percent change of false dependencies.

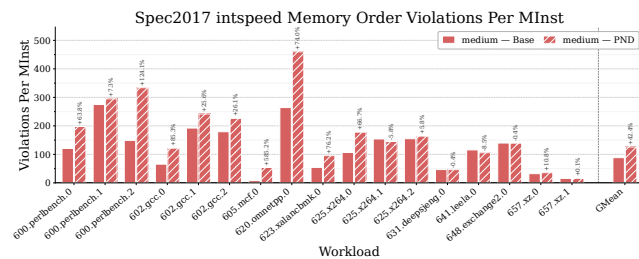


Figure 3: Percent change of memory order violations. Values roughly equal between core sizes.

620.*omnetpp_s* observes 6 million. This suggests due to the nature of the workload, a longer instruction window captures more dependencies which the larger Store Set tables cannot offset.

In select cases PND labels can lead to a performance regression, notably for input 2 of 600.*perlbench_s*. This is because loads have been mis-labelled from the train inputs and cause an increase in memory order violations on the test inputs, as seen in Figure 3 for the medium core configuration. Although memory order violations are increased in all workloads, they're mostly offset by the reduction in false dependencies. It is also worth noting that violations are already very rare events, measured in mega-instructions rather than kilo-instructions, so even a 100% increase can often still keep them in negligible ranges compared to the number of false dependencies. In input 2 of 600.*perlbench_s* though the absolute number of violations is at it's highest, and so offsets any reductions in false dependencies (which as was also already discussed have more a limited impact in this workload).

620.omnetpp_s sees the largest increase in violations yet maintains a performance gain for the medium configuration. This is possibly because it observes almost double the average intspeed dcache miss rate (4.3M vs 2.6M), leading to costly false dependencies that stall on stores long-to-resolve stores, but low penalty violations that occur during cache misses.

The degree to which violations increase is controlled by the threshold value discussed in Section ?? . The optimal threshold may still provide a net performance gain while still causing small regressions in specific workloads, and a larger threshold could instead be chosen to ensure regressions never occur. They could be also mitigated by specialising the PGO technique to each benchmark rather than the processor.

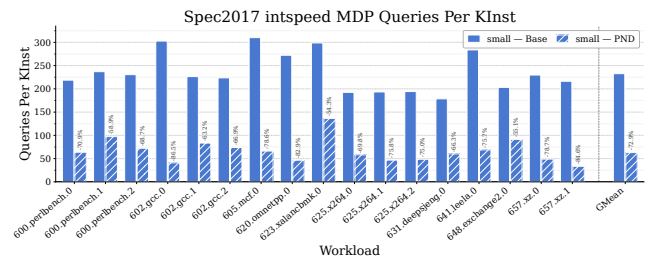


Figure 4: Percent change of MDP queries. Values roughly equal between core sizes.

5.3 Power

- analyse power savings, correlated to lookup reduction. small core has small shift in MDP power, but total core power saving congruent to IPC gain, which is an opportunity to remind how the gains come at no area/power tradeoff.

- need large core results which have a larger MDP power saving, but next to no core change. can try and speculate how this offsets growing MDP power for the future

- need large core with big phast power savings which will hopefully nudge total core power a little more? not sure where to go with this section if it doesn't tho.

5.4 Read Port Impact Estimation

Real processors have a limited number of read ports for each predictor component. If the memory dependence predictor has two ports, it can dispatch up to two memory operations per cycle. Gem5 does not model this behaviour by default, as it is a low-level detail that is difficult to represent accurately at the level of abstraction it simulates. Given the importance of this aspect to our proposed technique, we estimate the potential impact on IPC when read ports are limited. We generously assume that predictions always arrive within a single cycle for both XS Store Sets and PHAST. We assign a number of read ports to the memory dependence predictor for each processor model, as listed in Table 1. Dispatch is modelled to block when the number of memory operations dispatched in a cycle exceeds the number maximum number of read ports. This includes both loads and stores for XS Store Sets and only loads for PHAST. The count of in-use ports resets to zero each cycle. This provides a rough estimation of the impact of predicted non-dependent loads on workloads with a high density of memory operations. For clarity, **the number of read ports does not affect any other results presented in this paper.**

- insert results here

5.5 Profile Analysis & Threshold Sensitivity

- need to generate profile from test input for this, will take a few days

5.6 Store Set Table Size Sensitivity

As seen in Section 5.2, the number of false dependencies falls sharply when increasing the Store Set table sizes along with core size. We present a sensitivity study for how gmean IPC improvement and

base false dependencies in the small core configuration varies as both SSIT and LFST sizes grow.
- graphs here

Figure ?? shows that the majority of false dependencies can be removed by growing the Store Set tables, and likewise Figure ?? shows that the IPC gain diminishes along with this too. This demonstrates the main source of performance our technique offers is due to reducing hash collisions in the predictor tables, but without having to grow them.

Furthermore, we perform an iso-area comparison in the small core configuration between doubling the Store Set size verses growing the load queue by an equal number of bits. Whereas growing Store Sets provide an additional x% gmean IPC, the larger load

queue provides y%. This suggests that Store Set hash collisions would not typically be eliminated in small processors by growing the predictor size.

6 Threats to Validity

- encoding space (cite cooperative memory disambig)
- intended for specific target compilation

7 Future Work

- per benchmark tuning -serverless workloads
- JITs

8 Conclusion